

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CSA16 **COURSE CODE:** Data Warehousing and Data Mining

COURSE OUTCOME COVERED

CO5: Apply association rule mining techniques to discover interesting relationships and patterns within large datasets
Assessment Tool Weightage: 35%

QUESTIONS: 05

Total Marks:50

Annexure A

Pseudocode Writing Task (Algorithm Design)

Criteria	Problem Understanding (2)	Algorithm Design (3)	Use of Control Structures (2)	Pseudocode Structure (1)	Completeness (2)	Total(10)
Weightage	20%	30%	20%	10%	20%	100%
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						
<i>Q4</i>						
<i>Q5</i>						

Code of Conduct:

I A. LOKESH KUMAR (192524157) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Annexure A

Q1. Dataset: Online Shopping Transactions

Order ID	Products Purchased
O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger
O5	Mobile, Earphones

Question:

Design pseudocode for the **Apriori algorithm** to discover frequent product combinations from the online shopping dataset using a specified minimum support threshold. Clearly illustrate the candidate itemset generation and pruning process at each iteration. (BL3 – 1.05 Marks)

Q2. Dataset: Library Book Borrowing Records

Borrow ID	Books Borrowed
B1	Data Science, Python
B2	Python, Machine Learning
B3	Data Science, Machine Learning
B4	Data Science, Python, Machine Learning
B5	Python, Machine Learning

Question:

Develop pseudocode for the **FP-Growth algorithm** to identify frequent borrowing patterns without generating candidate itemsets. Explain the steps involved in constructing the FP-Tree using the given borrowing records. (BL4 – 1.05 Marks)

Q3. Dataset: Frequently Purchased Services

Service Combination	Support Count
{Internet}	4
{Streaming}	4
{Cloud Storage}	3
{Internet, Streaming}	3
{Streaming, Cloud Storage}	2

Question:

Write pseudocode to generate **association rules** from the given frequent service combinations. Include the procedures for confidence calculation, rule generation, and filtering based on a minimum confidence threshold. (BL3 – 1.05 Marks)

Q4. Dataset: Student Course Enrollment**Student ID Courses Enrolled**

S1	Python, Database
S2	Python, AI
S3	Database, AI
S4	Python, Database, AI
S5	Python

Question:

Design pseudocode for **constraint-based frequent itemset mining** where only itemsets containing the course "**Python**" are considered during the mining process. Explain how the constraint affects candidate generation and pruning. (BL4 – 1.05 Marks)

Q5. Dataset: Hospital Treatment Hierarchy**Patient ID Treatments Received**

P1	Healthcare, Cardiology, ECG
P2	Healthcare, Orthopedics
P3	Healthcare, Cardiology
P4	Healthcare, Cardiology, ECG
P5	Healthcare, Orthopedics

Question:

Develop pseudocode for **multilevel association rule mining** using the treatment hierarchy provided in the dataset. Explain how rules are generated at different hierarchy levels and how concept hierarchies improve pattern discovery.

1.

The online shopping dataset contains five orders over three products. The pseudocode below designs the Apriori algorithm to discover frequent product combinations for a specified minimum support threshold, and the trace that follows applies it to this dataset using an assumed minimum support of 50% (a support count of at least 3 out of 5 orders) purely to illustrate candidate generation and pruning concretely.

Pseudocode

```
ALGORITHM Apriori(Transactions T, minSupport)
1. k <- 1
2. C1 <- {each individual item in T}
3. L1 <- {items in C1 whose support >= minSupport}
4. WHILE L(k) is not empty DO
5.   k <- k + 1
6.   Ck <- CANDIDATE-GENERATE(L(k-1))    // join step
7.   FOR EACH candidate c in Ck DO
8.     FOR EACH subset s of c of size (k-1) DO
9.       IF s is NOT in L(k-1) THEN
10.        REMOVE c from Ck    // prune step (anti-monotone property)
11.     END FOR
12.   END FOR
13.   FOR EACH transaction t in T DO
14.     FOR EACH candidate c in Ck DO
15.       IF c is a subset of t THEN increment count(c)
16.   Lk <- {c in Ck : count(c) / |T| >= minSupport}
17. END WHILE
18. RETURN union of all Lk (all frequent itemsets)
```

Trace on the Online Shopping Dataset (minSupport = 50%, i.e. count >= 3 of 5)

Orders: O1={Mobile,Earphones}, O2={Mobile,Charger}, O3={Earphones,Charger},
O4={Mobile,Earphones,Charger}, O5={Mobile,Earphones}.

Iteration 1 (k = 1): Candidate Generation and Pruning

- C1 = {Mobile}, {Earphones}, {Charger}
- Counts: Mobile = 4 (O1,O2,O4,O5) = 80%; Earphones = 4 (O1,O3,O4,O5) = 80%;
Charger = 3 (O2,O3,O4) = 60%
- L1 = {Mobile}, {Earphones}, {Charger} — all three pass the 50% threshold, so no pruning occurs at this level.

Iteration 2 (k = 2): Candidate Generation and Pruning

- C2 (join of L1 with itself) = {Mobile,Earphones}, {Mobile,Charger},
{Earphones,Charger}

- Pruning check: every 1-item subset of each candidate is already in L1, so no candidate is eliminated by the join-step pruning rule before counting.

- Counts: {Mobile, Earphones} = 3 (O1, O4, O5) = 60% — frequent; {Mobile, Charger} = 2 (O2, O4) = 40% — infrequent, discarded; {Earphones, Charger} = 2 (O3, O4) = 40% — infrequent, discarded.

- L2 = {Mobile, Earphones} only.

Iteration 3 (k = 3): Termination

The join step requires at least two frequent (k-1)-itemsets that share a common (k-2)-prefix to generate a candidate k-itemset. Since L2 contains only one itemset, no valid 3-item candidate can be formed, so C3 is empty and the algorithm terminates.

Final Frequent Itemsets

{Mobile} (80%), {Earphones} (80%), {Charger} (60%), {Mobile, Earphones} (60%). The pruning step is what kept the algorithm efficient here — because {Mobile, Charger} and {Earphones, Charger} were eliminated at k = 2, the algorithm never needed to consider {Mobile, Earphones, Charger} at all, since Apriori's anti-monotone property guarantees that a 3-itemset cannot be frequent if any of its 2-item subsets is infrequent.

2.

The library borrowing dataset records five borrow transactions across three books. The pseudocode below builds the FP-Growth algorithm, which discovers frequent borrowing patterns without generating candidate itemsets by instead compressing the transaction database into an FP-Tree and mining that tree directly.

Pseudocode

ALGORITHM FPGrowth(Transactions T, minSupport)

1. Scan T once; compute support(item) for every item
2. $F \leftarrow \{\text{items with support} \geq \text{minSupport}\}$, sorted in descending order of support
3. Create root node of FP-Tree, labelled NULL
4. FOR EACH transaction t in T DO
5. Reorder items of t according to the order in F (drop non-frequent items)
6. INSERT-TREE(reordered t, root) // shares common prefixes as branches
7. Build a Header Table linking each item in F to all its occurrences in the tree
8. RETURN MINE-TREE(FP-Tree, Header Table, NULL, minSupport)

PROCEDURE MINE-TREE(Tree, HeaderTable, suffixPattern, minSupport)

9. FOR EACH item a_i in HeaderTable, starting from the least frequent DO
10. Form pattern $P = a_i \text{ UNION suffixPattern}$; output P as frequent
11. Build conditional pattern base for a_i by following node links
12. Build conditional FP-Tree from the conditional pattern base
13. IF conditional FP-Tree is not empty THEN
14. MINE-TREE(conditional FP-Tree, its HeaderTable, P, minSupport) // recursive
15. END FOR

Constructing the FP-Tree from the Borrowing Records (minSupport assumed = 40%, count ≥ 2 of 5)

Records: B1={Data Science,Python}, B2={Python,Machine Learning}, B3={Data Science,Machine Learning}, B4={Data Science,Python,Machine Learning}, B5={Python,Machine Learning}.

Step 1: Item Support and Frequency Ordering

- Python = 4 (B1,B2,B4,B5); Machine Learning = 4 (B2,B3,B4,B5); Data Science = 3 (B1,B3,B4) — all frequent at 40%.
- Descending order $F = [\text{Python, Machine Learning, Data Science}]$ (Python and Machine Learning tied at 4; Python listed first).

Step 2: Reordering Each Transaction by F

- B1: Data Science, Python -> reordered: Python, Data Science
- B2: Python, Machine Learning -> Python, Machine Learning
- B3: Data Science, Machine Learning -> Machine Learning, Data Science

- B4: Data Science, Python, Machine Learning -> Python, Machine Learning, Data Science
- B5: Python, Machine Learning -> Python, Machine Learning

Step 3: Inserting Transactions into the FP-Tree

- Insert B1 (Python, Data Science): root -> Python:1 -> Data Science:1
- Insert B2 (Python, Machine Learning): Python count becomes 2 -> new branch Machine Learning:1 under Python
- Insert B3 (Machine Learning, Data Science): no shared prefix with existing Python branch, so a new branch grows directly from root -> Machine Learning:1 -> Data Science:1
- Insert B4 (Python, Machine Learning, Data Science): Python becomes 3, Machine Learning (under Python) becomes 2, new Data Science:1 added under that Machine Learning node
- Insert B5 (Python, Machine Learning): Python becomes 4, Machine Learning (under Python) becomes 3

Resulting FP-Tree Structure

Root -> Python:4 -> {Data Science:1 (from B1), Machine Learning:3 -> Data Science:1 (from B4)}; Root -> Machine Learning:1 -> Data Science:1 (from B3). The Header Table links Python to its single node (count 4), links Machine Learning to two nodes (count 3 under Python, count 1 directly under root), and links Data Science to three separate nodes (one under each branch).

Why No Candidate Itemsets Are Generated

Unlike Apriori, FP-Growth never enumerates candidate 2-itemsets, 3-itemsets, and so on. Instead, mining proceeds by following each item's node links in the Header Table to build a small "conditional pattern base" (the prefix paths leading to that item), constructing a much smaller conditional FP-Tree from it, and recursing — for example, Data Science's conditional pattern base is {(Python:1), (Python,Machine Learning:1), (Machine Learning:1)}, from which frequent patterns involving Data Science are read directly off the compressed tree rather than being generated and tested as explicit candidates.

3.

This task starts from an already-computed table of frequent service combinations and their support counts, and requires pseudocode that generates association rules from these itemsets, computes confidence for each candidate rule, and filters out rules that fall below a minimum confidence threshold.

Pseudocode

```
ALGORITHM GenerateRules(FrequentItemsets L, minConfidence)
1. Rules <- empty set
2. FOR EACH frequent itemset F in L where |F| >= 2 DO
3.   FOR EACH non-empty proper subset A of F DO
4.     B <- F - A // remaining items form the consequent
5.     confidence <- support(F) / support(A)
6.     IF confidence >= minConfidence THEN
7.       ADD rule (A -> B) with its confidence to Rules
8.   END FOR
9. END FOR
10. RETURN Rules
```

Trace on the Given Frequent Service Combinations (minConfidence assumed = 70%)

Given: support(Internet)=4, support(Streaming)=4, support(Cloud Storage)=3,
support(Internet,Streaming)=3, support(Streaming,Cloud Storage)=2.

Candidate Rules from {Internet, Streaming}

- Internet -> Streaming: confidence = support(Internet,Streaming) / support(Internet) = $3/4 = 75\%$ -> passes filter ($\geq 70\%$)
- Streaming -> Internet: confidence = $3/4 = 75\%$ -> passes filter

Candidate Rules from {Streaming, Cloud Storage}

- Streaming -> Cloud Storage: confidence = support(Streaming,Cloud Storage) / support(Streaming) = $2/4 = 50\%$ -> filtered out (below 70%)
- Cloud Storage -> Streaming: confidence = $2/3 = 66.7\%$ -> filtered out (below 70%)

Final Output of GenerateRules at minConfidence = 70%

Only two rules survive filtering: Internet -> Streaming (75% confidence) and Streaming -> Internet (75% confidence). Both rules built on {Streaming, Cloud Storage} are discarded because their confidence falls short of the threshold, illustrating that a frequent itemset does not guarantee that every rule derivable from it will be strong — the filtering step in line 6 of the pseudocode is what enforces this distinction between merely frequent and genuinely reliable rules.

Why the Pseudocode Enumerates All Non-Empty Proper Subsets as Antecedents

Line 3 of the pseudocode deliberately considers every non-empty proper subset A of a frequent itemset F as a candidate antecedent, not just the single-item subsets. This matters because confidence is directional and asymmetric — $\text{support}(F)/\text{support}(A)$ is generally different from $\text{support}(F)/\text{support}(B)$ — so a rule can be strong in one direction and weak in the other, as seen above where Cloud Storage $\rightarrow$ Streaming (66.7%) is noticeably stronger than Streaming $\rightarrow$ Cloud Storage (50%) even though both come from the same frequent itemset. For a k-itemset, this generates $2^k - 2$ candidate rules (excluding the empty set and the full set as antecedent), which is small for the 2-itemsets in this dataset but is also why, in practice, rule generation exploits the fact that confidence is anti-monotone with respect to the size of the antecedent moving from a subset to the full itemset within a fixed consequent side, allowing additional pruning when generating rules from larger itemsets.

Strengthening the Filter with a Second Interestingness Measure (Lift)

Confidence alone can sometimes be misleading if the consequent is already very common on its own, so many implementations of GenerateRules add a secondary lift check, computed as $\text{lift}(A \rightarrow B) = \text{confidence}(A \rightarrow B) / \text{support}(B)$, alongside the minConfidence filter. Applying this to the two surviving rules: $\text{lift}(\text{Internet} \rightarrow \text{Streaming}) = 0.75 / (\text{support}(\text{Streaming}))$ — if the total number of transactions is taken as N and $\text{support}(\text{Streaming})$ as a fraction of N, lift greater than 1 indicates Internet and Streaming co-occur more often than would be expected if they were statistically independent, which further confirms that the Internet $\rightarrow$ Streaming and Streaming $\rightarrow$ Internet rules are not just individually confident but also genuinely positively correlated rather than being high-confidence purely because Streaming itself is common across most transactions.

Computational Consideration

Because the frequent itemsets are supplied directly in this task rather than needing to be mined first, the pseudocode's cost is dominated by the subset-enumeration loop in lines 2-8 rather than by database scans. For the small itemsets in this dataset (all of size 2), this cost is negligible, but the same structure scales to larger frequent itemsets discovered by Apriori or FP-Growth, where the number of candidate rules per itemset grows exponentially with itemset size — which is precisely why the minConfidence filter in line 6 is applied incrementally as each candidate rule is generated, rather than generating the full rule set first and filtering afterward, keeping memory usage bounded even when a frequent itemset is large.

4.

The student enrolment dataset lists courses taken by five students. The task requires constraint-based frequent itemset mining pseudocode that only considers itemsets containing the course "Python", and an explanation of how this item constraint changes candidate generation and pruning compared to unconstrained Apriori.

Pseudocode

```
ALGORITHM ConstrainedApriori(Transactions T, minSupport, requiredItem)
1. k <- 1
2. C1 <- {each individual item in T}
3. L1 <- {items in C1 whose support >= minSupport}
4. WHILE L(k) is not empty DO
5.   k <- k + 1
6.   Ck <- CANDIDATE-GENERATE(L(k-1))
7.   FOR EACH candidate c in Ck DO
8.     IF requiredItem is NOT in c THEN
9.       REMOVE c from Ck          // constraint-based pruning (pushed early)
10.    CONTINUE
11.   FOR EACH subset s of c of size (k-1) DO
12.     IF s is NOT in L(k-1) THEN REMOVE c from Ck // standard anti-monotone
pruning
13.   Count support of remaining candidates in Ck against T
14.   Lk <- {c in Ck : support(c) >= minSupport}
15. END WHILE
16. RETURN union of all Lk restricted to itemsets containing requiredItem
```

Trace on the Student Enrolment Dataset (minSupport assumed = 40%, count >= 2 of 5; requiredItem = "Python")

Records: S1={Python,Database}, S2={Python,AI}, S3={Database,AI},
S4={Python,Database,AI}, S5={Python}.

Iteration 1 (k = 1)

- Counts: Python = 4 (S1,S2,S4,S5) = 80%; Database = 3 (S1,S3,S4) = 60%; AI = 3 (S2,S3,S4) = 60%

- L1 = {Python}, {Database}, {AI} — all three pass minSupport, but only {Python} satisfies the required-item constraint on its own.

Iteration 2 (k = 2) — Constraint Applied Before Counting

- Unconstrained C2 would be {Python,Database}, {Python,AI}, {Database,AI}.
- The constraint check in lines 8-10 immediately discards {Database,AI} because it does not contain Python — its support is never even counted, unlike in standard Apriori, where it would be counted (Database,AI occurs in S3,S4 = 40%, and would normally survive as frequent).

- Remaining candidates: $\{\text{Python, Database}\} = 2$ ($S1, S4$) = 40% $\rightarrow$ frequent; $\{\text{Python, AI}\} = 2$ ($S2, S4$) = 40% $\rightarrow$ frequent.
- $L2 = \{\text{Python, Database}\}, \{\text{Python, AI}\}.$

Iteration 3 ($k = 3$)

The only candidate is $\{\text{Python, Database, AI}\}$, which does contain Python and so passes the constraint check; its support is $S4$ only = $1/5 = 20\%$, below the 40% threshold, so it is pruned by the standard anti-monotone rule and $L3$ is empty, ending the algorithm.

How the Constraint Affects Candidate Generation and Pruning

Pushing the "must contain Python" constraint directly into the candidate-generation loop (rather than filtering the final results afterward) means that itemsets like $\{\text{Database, AI}\}$ are eliminated before support counting is even attempted, saving a full pass over the transaction database for that candidate. This is a meaningful efficiency gain in constraint-based mining: instead of computing the complete frequent-itemset lattice and filtering at the end, the constraint actively shrinks the search space at every level, which matters increasingly as the number of distinct courses (items) grows and the unconstrained candidate space would otherwise expand combinatorially.

5.

The hospital treatment dataset follows an implicit concept hierarchy: Healthcare is the general top-level category, Cardiology and Orthopedics are specific departments beneath it, and ECG is a specific procedure beneath Cardiology. The task requires pseudocode for multilevel association rule mining using this hierarchy, along with an explanation of how rules differ across levels.

Pseudocode

```
ALGORITHM MultilevelMining(Transactions T, ConceptHierarchy H,
minSupportByLevel)
1. levels <- H.levels ordered from top (most general) to bottom (most specific)
2. FOR EACH level Lv in levels DO
3.   T_Lv <- generalize each transaction in T to the items that exist at level Lv
4.   (e.g., replace 'ECG' with its ancestor 'Cardiology' when mining at the Cardiology
level)
5.   minSup_Lv <- minSupportByLevel[Lv] // typically reduced at deeper levels
6.   F_Lv <- Apriori(T_Lv, minSup_Lv) // standard frequent-itemset mining at this
level
7.   Rules_Lv <- GenerateRules(F_Lv, minConfidence)
8.   FILTER Rules_Lv to remove rules that are direct ancestor-descendant restatements
9.   (e.g., discard 'X -> Healthcare' since Healthcare is an ancestor of every item)
10.  OUTPUT Rules_Lv
11. END FOR
```

Trace on the Hospital Treatment Hierarchy

Hierarchy: Healthcare (level 1) -> {Cardiology, Orthopedics} (level 2) -> {ECG} (level 3, child of Cardiology). Records: P1={Healthcare,Cardiology,ECG}, P2={Healthcare,Orthopedics}, P3={Healthcare,Cardiology}, P4={Healthcare,Cardiology,ECG}, P5={Healthcare,Orthopedics}.

Level 1 (Healthcare)

Healthcare has support $5/5 = 100\%$, since every patient receives it. Any rule involving Healthcare (e.g., Cardiology -> Healthcare) would have 100% confidence for every antecedent, so lines 8-9 of the pseudocode filter these out as trivial ancestor restatements rather than reporting them as discoveries.

Level 2 (Cardiology, Orthopedics)

- Cardiology support = $3/5 = 60\%$ (P1,P3,P4); Orthopedics support = $2/5 = 40\%$ (P2,P5).
- No patient receives both Cardiology and Orthopedics, so no meaningful 2-itemset rule exists between them at this level — this itself is a useful finding, indicating the two departments serve disjoint patient groups in this dataset.

Level 3 (ECG, under Cardiology)

- ECG support = $2/5 = 40\%$ (P1,P4); {Cardiology, ECG} support = $2/5 = 40\%$ (P1,P4).

- $\text{Confidence}(\text{Cardiology} \rightarrow \text{ECG}) = 2/3 = 66.7\%$
- $\text{Confidence}(\text{ECG} \rightarrow \text{Cardiology}) = 2/2 = 100\%$

These level-3 rules are the most specific and actionable output of the whole mining process: they show that every patient who receives an ECG is a Cardiology patient (100% confidence), while about two-thirds of Cardiology patients specifically receive an ECG.

How Concept Hierarchies Improve Pattern Discovery

Mining a single flat level would either bury useful patterns under the overwhelming, trivial support of top-level items like Healthcare, or miss them entirely if only the most specific level (ECG) were mined in isolation without connecting it back to its parent Cardiology. By generalising transactions level by level and applying a reduced minimum support threshold as the hierarchy gets more specific (since deeper items naturally have lower support), multilevel mining surfaces both broad departmental patterns (Level 2) and precise procedure-level associations (Level 3), while the ancestor-restatement filter in the pseudocode prevents the output from being cluttered with rules that add no real analytical value.